

C++ Copy Constructor

A Copy constructor is an **overloaded** constructor used to declare and initialize an object from another object.

Copy Constructor is of two types:

Default Copy constructor: The compiler defines the default copy constructor. If the user defines no copy constructor, compiler supplies its constructor.

User Defined constructor: The programmer defines the user-defined constructor.

Syntax Of User-defined Copy Constructor:

1. Class_name(**const** class_name &old_object);

Consider the following situation:

```
class A
{
    A(A &x) // copy constructor.
    {
        // copyconstructor.
    }
}
```

In the above case, **copy constructor can be called in the following ways:**

Let's see a simple example of the copy constructor.

```

using namespace std;

class A
{
public:
    int x;
    A(int a)           // parameterized constructor.
    {
        x=a;
    }
    A(A &i)           // copy constructor
    {
        x = i.x;
    }
};

int main()
{
    A a1(20);         // Calling the parameterized constructor.
    A a2(a1);         // Calling the copy constructor.
    cout<<a2.x;
    return 0;
}

```

When Copy Constructor is called

Copy Constructor is called in the following scenarios:

- When we initialize the object with another existing object of the same class type. For example, Student s1 = s2, where Student is the class.
- When the object of the same class type is passed by value as an argument.
- When the function returns the object of the same class type by value.
-
- **Two types of copies are produced by the constructor:**
 - Shallow copy
 - Deep copy

Shallow Copy

- The default copy constructor can only produce the shallow copy.
- A Shallow copy is defined as the process of creating the copy of an object by copying data of all the member variables as it is.

Let's understand this through a simple example:

```
class Demo
{
    int a;
    int b;
    int *p;
public:
    Demo()
    {
        p=new int;
    }
    void setdata(int x,int y,int z)
    {
        a=x;
        b=y;
        *p=z;
    }
    void showdata()
    {
        std::cout << "value of a is : " <<a<< std::endl;
        std::cout << "value of b is : " <<b<< std::endl;
        std::cout << "value of *p is : " <<*p<< std::endl;
    }
}
```

```

int main()
{
    Demo d1;
    d1.setdata(4,5,7);
    Demo d2 = d1;
    d2.showdata();

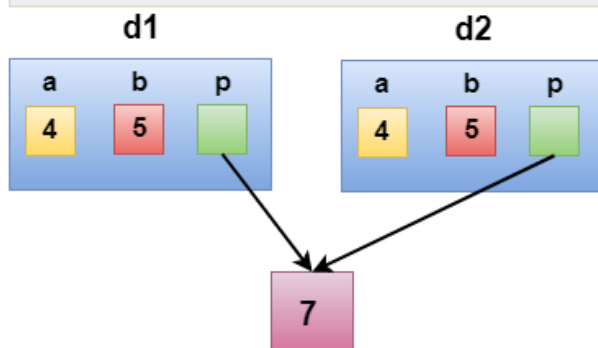
    return 0;
}

```

```

value of a is : 4
value of b is : 5
value of *p is : 7

```



In the above case, a programmer has not defined any constructor, therefore, the statement **Demo d2 = d1;** calls the default constructor defined by the compiler. The default constructor creates the exact copy or shallow copy of the existing object. Thus, the pointer p of both the objects point to the same memory location. Therefore, when the memory of a field is freed, the memory of another field is also automatically freed as both the fields point to the same memory location. This problem is solved by the **user-defined constructor** that creates the **Deep copy**.

Deep copy

Deep copy dynamically allocates the memory for the copy and then copies the actual value, both the source and copy have distinct memory locations. In this way, both the source and

copy are distinct and will not share the same memory location. Deep copy requires us to write the user-defined constructor.

Let's understand this through a simple example.

```
#include <iostream>
using namespace std;
class Demo
{
public:
    int a;
    int b;
    int *p;

    Demo()
    {
        p=new int;
    }
    Demo(Demo &d)
    {
        a = d.a;
        b = d.b;
        p = new int;
        *p = *(d.p);
    }
}
```

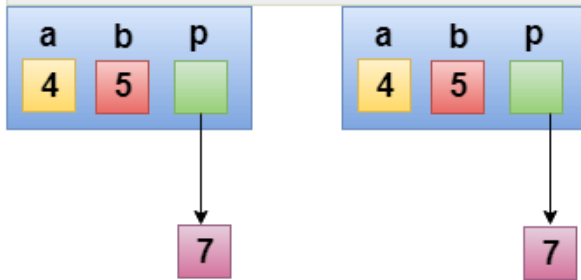
```
void setdata(int x,int y,int z)
{
    a=x;
    b=y;
    *p=z;
}

void showdata()
{
    std::cout << "value of a is : " <<a<< std::endl;
    std::cout << "value of b is : " <<b<< std::endl;
    std::cout << "value of *p is : " <<*p<< std::endl;
}

};

int main()
{
    Demo d1;
    d1.setdata(4,5,7);
    Demo d2 = d1;
    d2.showdata();
    return 0;
}
```

```
value of a is : 4  
value of b is : 5  
value of *p is : 7
```



In the above case, a programmer has defined its own constructor, therefore the statement **Demo d2 = d1;** calls the copy constructor defined by the user. It creates the exact copy of the value types data and the object pointed by the pointer p. Deep copy does not create the copy of a reference type variable.

ADVERTISEMENT

Differences b/w Copy constructor and Assignment operator(=)

Copy Constructor	Assignment Operator
It is an overloaded constructor.	It is a bitwise operator.
It initializes the new object with the existing object.	It assigns the value of one object to another object.
Syntax of copy constructor: Class_name(const class_name &object_name) { // body of the constructor. }	Syntax of Assignment operator: Class_name a,b; b = a;
○ The copy constructor is invoked when the new object is initialized with the existing object. ○ The object is passed as an argument to the function. ○ It returns the object.	The assignment operator is invoked when we assign the existing object to a new object.
Both the existing object and new object shares the different memory locations.	Both the existing object and new object shares the same memory location.
If a programmer does not define the copy constructor, the compiler will automatically generate the implicit default copy constructor.	If we do not overload the "=" operator, the bitwise copy will occur.